

MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism

发表：2025/04/03

♥ 管理收藏夹

⚡ TL;DR 与摘要

🗺 思维导图

🖼 论文图片分析

🔍 相似文献推荐

PDF

原文

精读

内容目录

展开目录 ▼

1. 论文基本信息

1.1. 标题

MegaScale-Infer: 通过解耦专家并行实现大规模混合专家服务

1.2. 作者

Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, Xin Liu

研究背景与隶属机构：作者主要来自 **ByteDance Seed**（字节跳动种子团队）和 **Peking University**（北京大学）。这是一项工业界与学术界紧密合作的研究，旨在解决实际生产环境中大规模模型服务面临的挑战。

1.3. 发表期刊/会议

arXiv 预印本 (arXiv:2504.02263)。截至分析时间，该论文尚未正式发表于特定期刊或会议，但作为来自字节跳动和北大的系统性工作，其内容涉及高性能计算和系统架构，具有较高的工程参考价值和学术关注度。

1.4. 发表年份

2025年 (根据UTC时间戳 2025-04-03)

1.5. 摘要

研究目的：随着大语言模型（LLM）向大规模发展，混合专家模型因其能以较低的计算复杂度扩展模型参数而备受关注。然而，MoE 的稀疏激活特性在推理阶段导致前馈网络（FFN）从计算密集型转变为内存密集型，造成 GPU 利用率低下和运营成本高昂。

核心方法：论文提出了 **MegaScale-Infer**，一个高效且经济的大规模 MoE 模型服务系统。

MegaScale-Infer 将模型层内的注意力模块和 FFN 模块进行**解耦**，使得两者可以独立扩展、采用定制化的并行策略，并支持异构硬件部署。为了充分利用解耦架构并应对 MoE 的稀疏性，系统引入了**乒乓流水线并行**，将请求批次划分为微批次，在注意力节点和 FFN 节点之间往复传递以隐藏通信开销。此外，针对解耦模块间的数据传输（如词元分发），开发了高性能 **M2N 通信库**，消除了不必要的 GPU-to-CPU 数据拷贝和同步开销。

主要结果：实验结果表明，MegaScale-Infer 实现了比现有最先进解决方案高达 **1.90 倍** 的单 GPU 解码吞吐量提升。

1.6. 原文链接

论文链接: <https://arxiv.org/abs/2504.02263> PDF 链接: <https://arxiv.org/pdf/2504.02263v4> 状态: 预印本

2. 整体概括

2.1. 研究背景与动机

核心问题：大语言模型（LLM）的参数规模正呈指数级增长。混合专家架构通过稀疏激活机制，允许模型在不线性增加计算量的情况下扩展参数量，是构建超大规模模型的有效手段。然而，在推理服务阶段，MoE 的稀疏性带来了新的挑战：

- 1. GPU 利用率低下：**在解码阶段，注意力模块因为需要访问大量 KV 缓存而受限于内存带宽；FFN 模块在稠密模型中可以通过大批次请求达到计算饱和，但在 MoE 中，由于每个专家只处理一小部分词元，导致 FFN 的有效批次大小急剧下降，使其也变成了受限于内存的操作，无法充分利用 GPU 的计算能力。
- 2. 成本高昂：**为了维持吞吐量，往往需要部署更多的 GPU，导致运营成本显著增加。

现有挑战与空白：现有的 LLM 服务系统（如 vLLM, TensorRT-LLM）通常将整个模型层（注意力 + FFN）部署在同一个 GPU 上。虽然 Infinite-LLM 等工作尝试了解耦注意力，但主要针对长上下文

场景，未能有效解决 MoE 稀疏性导致的 FFN 计算利用率问题。

切入点与创新思路： MegaScale-Infer 的核心思路是**彻底解耦注意力模块和 FFN 模块**。通过将它们部署在不同的 GPU 上，可以独立地扩展这两部分资源。具体来说，可以复制多个注意力节点来聚合请求，从而增大发送给 FFN 专家的批次大小，使 FFN 重新变为计算密集型任务。同时，这种解耦允许针对注意力（内存密集）和 FFN（计算密集）的特性选择不同类型的 GPU（异构部署），以最大化性价比。

2.2. 核心贡献/主要发现

主要贡献：

1. **解耦专家并行架构：** 提出了一种新的并行策略，将 Transformer 层内的注意力和 FFN 分离到不同的计算节点。这不仅允许独立扩展，还支持针对不同模块特性的异构硬件部署（如用高带宽 GPU 跑注意力，用高算力 GPU 跑 FFN）。
2. **乒乓流水线并行：** 设计了一种针对解耦架构的流水线策略，通过微批次在注意力节点和专家节点之间乒乓传输，有效隐藏了节点间通信开销，并保持了 GPU 的高利用率。
3. **高性能 M2N 通信库：** 针对解耦后 M 个注意力节点与 N 个专家节点之间任意通信模式（M2N），开发了一套高性能通信库。该库消除了传统通信库（如 NCCL）中的 GPU-CPU 拷贝、组初始化开销和 GPU 同步等待，显著降低了延迟并提升了稳定性。
4. **性能与成本效益：** 在实验中，MegaScale-Infer 在同构集群上实现了高达 1.90 倍的吞吐量提升，在异构集群上实现了 1.86 倍的单位成本吞吐量提升。

关键结论： 通过算法与系统的协同设计，特别是利用解耦架构和定制化的通信机制，可以克服 MoE 稀疏性带来的推理效率瓶颈，实现高性能且低成本的大规模模型服务。

3. 预备知识与相关工作

3.1. 基础概念

为了理解本文，读者需要掌握以下核心概念：

- **混合专家：** 这是一种模型架构，它不使用单一巨大的前馈网络（FFN），而是使用多个较小的 FFN（称为“专家”）。模型包含一个门控网络，根据输入词元的特征动态选择最相关的 top-k 个专家来处理该词元。这使得模型可以在增加参数量的同时，保持每次推理的计算量相对恒定（稀疏激活）。
- **LLM 推理的两个阶段：**
 - **预填充：** 处理输入提示词的阶段。模型需要计算输入词元之间的两两注意力，计算量随序列长度平方增长，通常是计算密集型的。

- **解码：** 逐个生成输出词元的阶段。每生成一个新词元，需要计算它与之前所有词元的注意力。由于需要反复读取存储在内存中的 KV 缓存，这个阶段通常是内存带宽密集型的。
- **KV 缓存：** 在自回归生成过程中，为了避免重复计算，模型会将每个词元的键和值向量存储下来。在生成后续词元时，直接从缓存中读取这些向量，大大减少了计算量，但也带来了巨大的内存访问需求。
- **张量并行：** 一种模型并行技术，将模型参数（如矩阵）切分到多个 GPU 上。每个 GPU 只计算矩阵的一部分，然后通过通信（如 All-Reduce）合并结果。这通常用于加速单个算子（如矩阵乘法），但会引入较高的通信开销，因此常限于单机多卡场景。
- **专家并行：** 专门针对 MoE 模型的并行技术。每个 GPU 上只存放一部分专家。当词元需要路由到特定专家时，需要通过 All-to-All 通信将词元发送到对应的 GPU 上。
- **流水线并行：** 将模型的不同层分配到不同的 GPU 上。数据像流水线一样流经各个 GPU。虽然可以处理更大的模型，但会增加流水线启动和排空的气泡，以及层间通信的开销。

3.2. 前人工作

作者在相关工作部分提到了以下关键研究：

- **LLM 服务优化：**
 - **vLLM [51]：** 提出了 PagedAttention 技术，高效管理 KV 缓存内存，是当前流行的 LLM 服务框架。
 - **Orca [79]：** 引入了迭代级调度以提高吞吐量。
 - **Sarathi-Serve [23]：** 通过分块处理预填充请求并与解码请求混合批处理，解决吞吐-延迟权衡。
 - 这些工作主要关注稠密模型或通用的 KV 缓存管理，未针对 MoE 的稀疏性进行专门的架构解耦。
- **资源解耦：**
 - **Infinite-LLM [54]：** 将注意力模块解耦出来以支持长上下文。但它主要解决内存容量限制，通信模式相对简单，不如 MoE 中的 Top-k 路由复杂，因此对 MoE 推理优化效果有限。
 - **DistServe [82]：** 解耦了预填充和解码阶段。MegaScale-Infer 借鉴了这一思想，并将其进一步应用到解码阶段内部的注意力和 FFN 模块解耦。
- **MoE 优化：**
 - **DeepSpeed-MoE [62]：** 提出了专家并行技术，用于 MoE 的训练和推理。
 - **Tutel [46]：** 提供了自适应的 MoE 实现。
 - **DeepEP [5]：** DeepSeek 提出的针对专家并行的通信库，利用 GPU 直接进行 GPU-to-GPU 通信。MegaScale-Infer 的 M2N 库与其不同，采用 CPU 辅助通信以避免占用 GPU 计算资源。

3.3. 技术演进

该领域的技术演进路径可以概括为：

1. **单体部署**：早期将整个模型放在单个或少量 GPU 上，受限于显存。
2. **模型并行 (TP/PP)**：为了突破单卡显存限制，引入了张量并行和流水线并行。
3. **阶段解耦**：发现预填充（计算密集）和解码（内存密集）特性差异巨大，因此将这两个阶段分离到不同集群。
4. **模块解耦 (本文)**：MegaScale-Infer 进一步深入到 Transformer 层内部，解耦注意力和 FFN，以应对 MoE 稀疏性带来的特殊负载不均衡问题。

3.4. 差异化分析

与现有工作相比，MegaScale-Infer 的核心区别在于：

- **粒度更细的解耦**：不同于仅解耦预填充/解码阶段，本文解耦了层内的计算模块。
- **针对 MoE 的流水线**：传统的流水线并行用于切分层，而本文的“乒乓流水线”用于在解耦的注意力和 FFN 之间传递微批次，目的是隐藏通信而非切分模型。
- **通信模式创新**：针对 M-to-N 的动态路由通信，开发了专门的通信库，而非直接复用通用的 All-to-All 原语。

4. 方法论

4.1. 方法原理

MegaScale-Infer 的核心原理是**利用计算与通信的重叠来消除解耦架构带来的额外开销**。通过将注意力（受限于内存）和 FFN（受限于计算）分离，我们可以针对它们各自的瓶颈（内存带宽 vs 计算能力）配置最优的硬件资源。然而，分离意味着数据需要在网络间传输。为了不让通信成为瓶颈，系统设计了一种“乒乓”机制，让计算节点在处理当前批次数据的同时，网络传输下一个批次的数据，从而实现计算与通信的完全重叠。

4.2. 核心方法详解

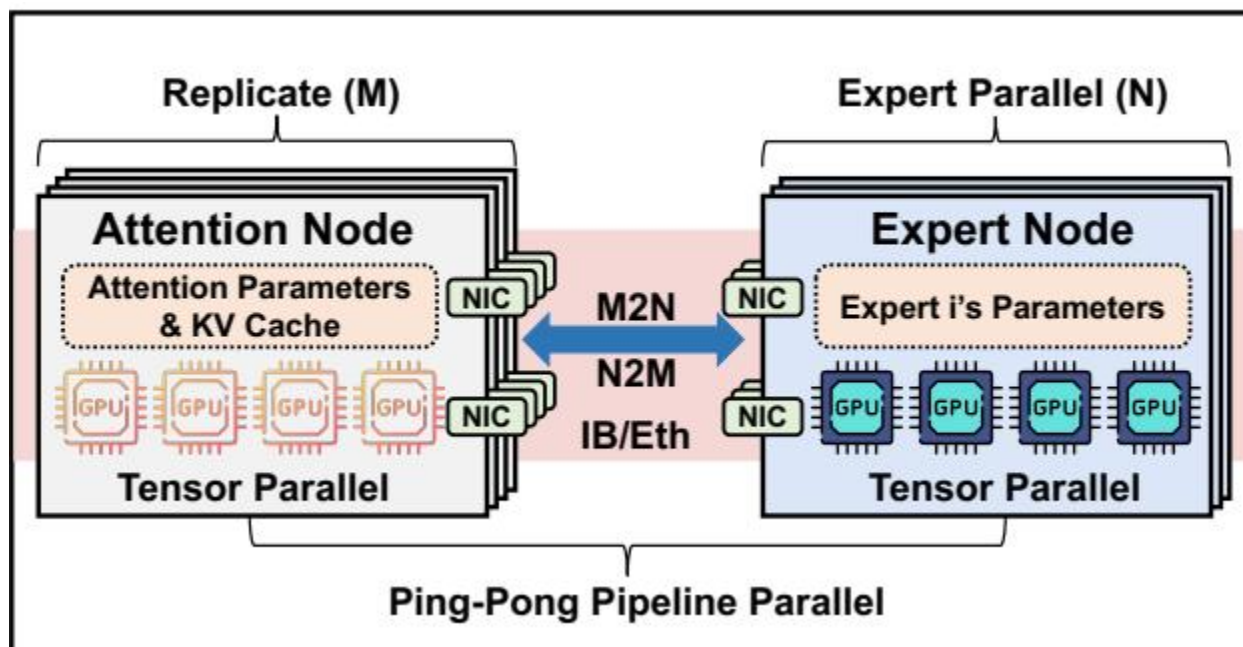
4.2.1. 解耦专家并行架构

MegaScale-Infer 将一个 Transformer 层拆分为两部分：

- 注意力节点：**负责计算 Attention 模块。由于解码阶段主要受限于 KV 缓存访问，这部分需要高内存带宽。系统可以部署多个注意力节点副本（数据并行），每个副本处理一部分请求，然后将结果汇总发送给专家节点。
- 专家节点：**负责计算 MoE（FFN）模块。这部分需要高计算算力。每个专家节点存储一部分专家参数。

这种架构允许独立扩展。例如，如果有 8 个注意力节点和 8 个专家，那么每个专家实际上接收到的有效批次大小是单个注意力节点批次的 8 倍（假设路由均匀）。这极大地提升了 FFN 的计算密度。

下图（原文 Figure 3）展示了 MegaScale-Infer 的运行实例架构，清晰地描绘了注意力节点与专家节点通过 M2N 通信进行交互的解耦结构：



该图像是MegaScale-Infer运行实例架构的示意图。图中展示了注意力节点和专家节点之间的比例复制和专家并行配置，以及通过ping-pong管道并行实现的微批次处理。注意力节点与专家节点之间的数据传输通过高性能的M2N通信库进行，优化了GPU利用率。

4.2.2. 乒乓流水线并行

由于解耦，一个词元必须先经过注意力节点，再经过专家节点。如果简单地按顺序处理，当一个节点在计算时，另一个节点和网络链路就会闲置。为了解决这个问题，MegaScale-Infer 引入了乒乓流水线。

具体流程：系统将一个全局批次 B 划分为 m 个微批次。注意力节点和专家节点并非同步处理同一个微批次，而是像打乒乓球一样交替处理。当注意力节点处理微批次 i 时，专家节点可能在处理微批次 $i-1$ ，同时网络正在传输微批次 $i-2$ 的数据。

性能约束与公式分析：为了使流水线高效运转，论文定义了以下关键变量和约束条件：设 T_a 为注意力节点处理一个微批次的计算时间， T_e 为专家节点处理一个微批次的计算时间。定义 $T_f =$

$\max\{T_a, T_e\}$ 为两者中的较大值。设 T_c 为单次双向通信（注意力->专家 或 专家->注意力）的时间。

为了达到计算与通信的完美重叠，必须满足以下约束条件：

$$T_a \approx T_e,$$

$$T_c < T_f,$$

$$m \times T_f \geq 2 \times (T_f + T_c).$$

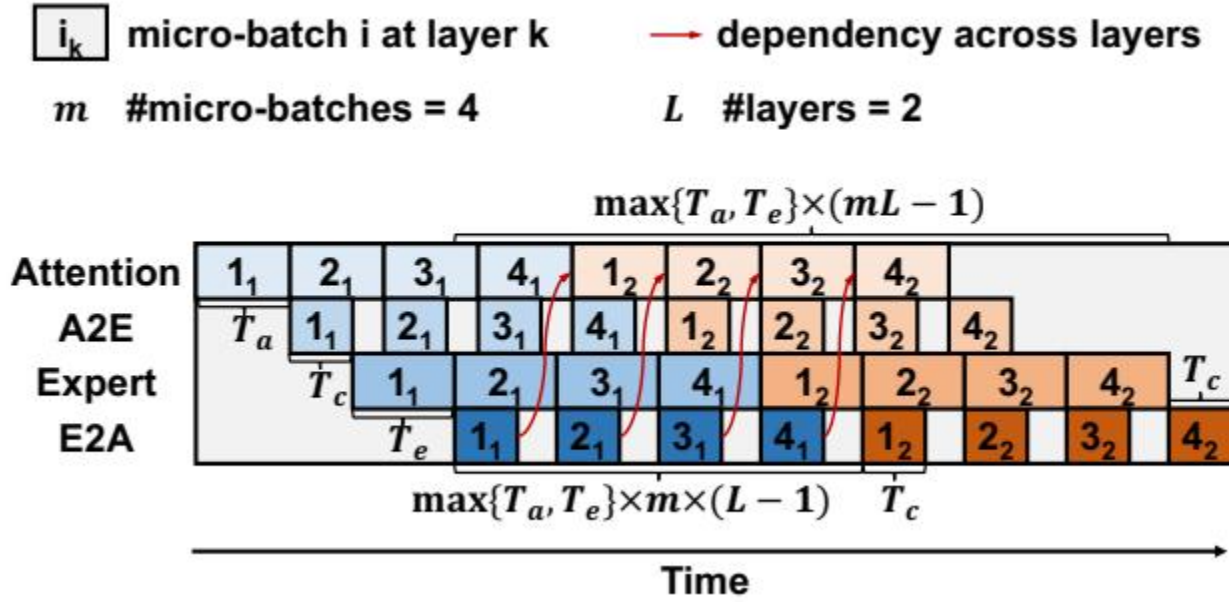
符号解释：

- T_a : Attention 模块计算一个微批次的时间。
- T_e : FFN (Expert) 模块计算一个微批次的时间。
- T_f : T_a 和 T_e 中的最大值，即瓶颈阶段的计算时间。
- T_c : 单次 M2N 通信的时间。
- m : 微批次的数量。

公式含义解析：

1. **约束 1 ($T_a \approx T_e$):** 要求注意力节点和专家节点的计算时间尽可能接近。如果相差太大，快的节点在等待慢的节点时会产生空闲时间。系统通过调整注意力节点的数量（数据并行度）来平衡这两者的负载。
2. **约束 2 ($T_c < T_f$):** 要求通信时间必须小于瓶颈计算时间。只有通信足够快，才能被计算时间完全掩盖，否则通信会成为新的瓶颈。
3. **约束 3 ($m \times T_f \geq 2 \times (T_f + T_c)$):** 这是流水线充满的条件。不等式左边是处理完整个全局批次所需的总时间（理想流水线状态下），右边是处理前两个微批次并完成它们通信所需的时间（流水线启动阶段）。只有当流水线稳态阶段的时间足以覆盖启动阶段的开销时，流水线才是高效的。由此可推导出最小微批次数量 $m \geq 2 \times (1 + \frac{T_c}{T_f})$ 。

下图（原文 Figure 4）形象地展示了乒乓流水线并行的工作原理，微批次在 Attention 和 Expert 之间交替流动，实现了计算与通信的重叠：



延迟模型： 基于上述流水线，论文给出了单个解码迭代（处理一层）的延迟估算公式。对于单个微批次，其迭代延迟 T_{iter} 满足：

$$(T_a + T_e + 2T_c) + mT_f(L - 1) \leq T_{iter} \leq mT_fL.$$

符号解释：

- L : 模型的层数。
- 该公式表示，第一个微批次需要经历完整的启动开销 $(T_a + T_e + 2T_c)$ ，后续的 $L-1$ 层可以利用流水线并行，每层耗时约 mT_f 。

整个全局批次的总延迟 T_{total} 为：

$$T_{total} = (T_a + T_e + 2T_c) + T_f(mL - 1).$$

4.2.3. 部署计划搜索

为了找到最优的系统配置（如注意力节点数量、专家节点 TP 大小、微批次数量等），MegaScale-Infer 使用了一个基于性能模型的搜索算法（Algorithm 1）。

搜索空间与目标： 算法遍历可能的配置组合，包括：

- 注意力节点的张量并行度 tp_a 。
- 专家节点的张量并行度 tp_e 。
- 微批次数量 m 。
- 全局批次大小 B 。

目标是最大化**单位成本吞吐量**，同时满足延迟 SLO（Service Level Objective）约束。

性能模拟： 为了快速评估配置，系统建立了一个性能模型来估算 T_a, T_e, T_c 。

- T_a 和 T_e : 主要由 GEMM (矩阵乘法) 操作决定。模型考虑了算术强度和 KV 缓存访问开销。例如, T_a 模型为 $k_1 b_a + k_2$, 其中 b_a 是注意力节点的批次大小, k_1, k_2 是通过 profiling 得到的系数。
- T_c : 通信时间取决于网络带宽和消息大小。模型考虑了网络带宽利用率与消息大小的函数关系。

$$T_c = \max\left\{\frac{b_a h K / tp_a}{W_a \times Util(b_a h K / tp_a)}, \frac{b_e h / tp_e}{W_e \times Util(b_e h / tp_e)}\right\},$$

符号解释:

- b_a, b_e : 注意力和专家节点的微批次大小。
- h : 模型的隐藏层维度。
- K : 选择的专家数量。
- tp_a, tp_e : 注意力和专家节点的张量并行度。
- W_a, W_e : 注意力和专家节点的网络带宽。
- $Util(\cdot)$: 网络带宽利用率函数, 随消息大小变化。

此外, 搜索过程还需满足 GPU 内存容量约束:

$$4mb_a shL/g + 2P_a < tp_a C_a.$$

符号解释:

- s : 平均序列长度。
- g : GQA (Grouped Query Attention) 中每组查询头的数量。
- P_a : 注意力模块的参数大小。
- C_a : 注意力 GPU 的显存容量。
- 该公式确保 KV 缓存和模型参数能放入显存。

4.2.4. 异构部署

解耦架构带来的另一个优势是支持异构硬件。系统可以根据模块特性选择最经济的 GPU:

- **注意力节点**: 内存密集型, 适合使用显存大、带宽高但算力相对便宜的 GPU (如 NVIDIA H20)。
- **专家节点**: 计算密集型, 适合使用算力强、性价比高的 GPU (如 NVIDIA L40S)。

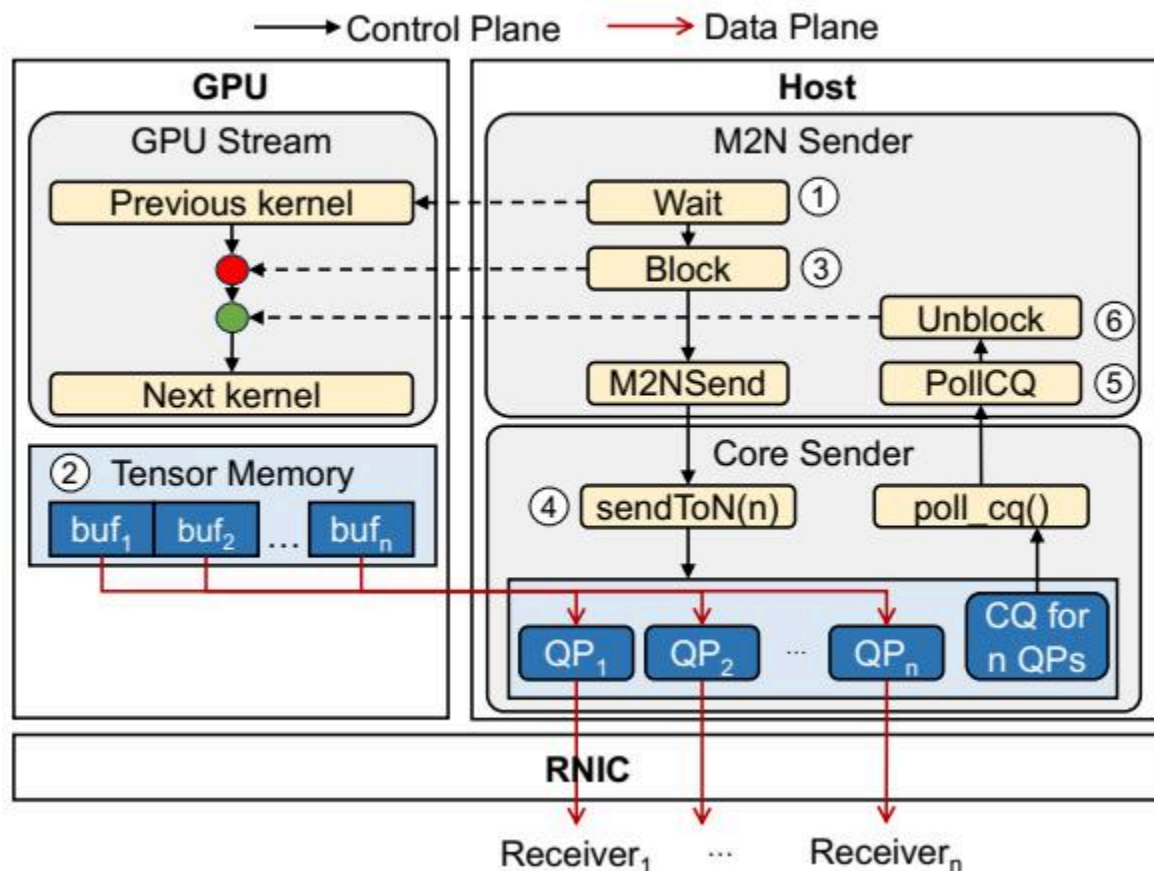
论文通过枚举不同 GPU 组合 (如 H20+L40S), 计算其单位成本吞吐量, 来选择最优的硬件部署方案。

4.2.5. 高性能 M2N 通信库

在解耦架构中，M 个注意力节点需要将词元发送给 N 个专家节点，这是一种动态的 M2N 通信模式。现有的通用通信库 NCCL 在此场景下存在高额外开销和尾部延迟不稳定的问题。

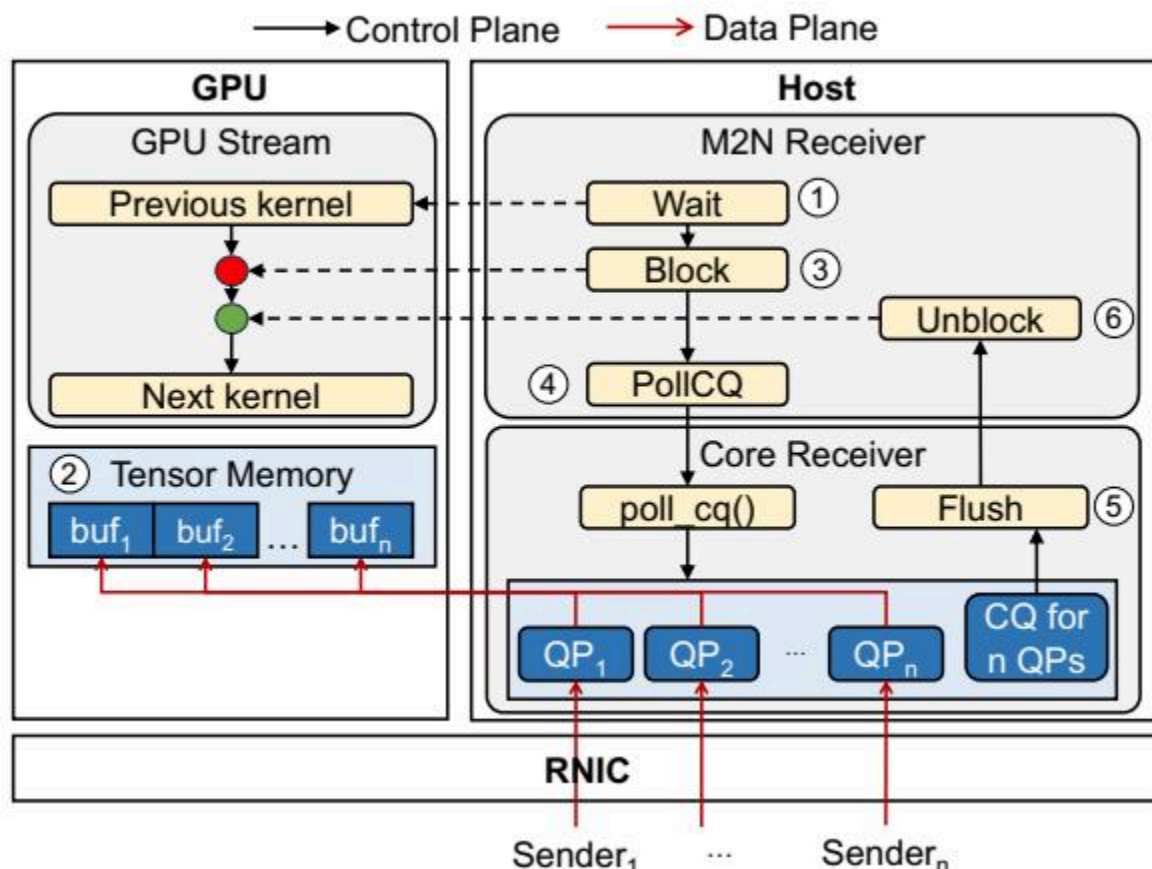
M2N Sender 设计： 下图（原文 Figure 6）展示了 M2N Sender 的组件交互。发送端采用 CPU 代理的方式处理网络传输，避免占用 GPU 计算资源。

1. **等待 GPU 数据：** 使用 CUDA Events 等待 GPU 计算完成，确保待发送张量已准备好。
2. **阻塞 CUDA 流：** 防止后续内核覆盖发送缓冲区。
3. **RDMA 传输：** 使用自研的 CPU 通信库，通过 RDMA Write 直接将数据写入远程 GPU 内存。
4. **轮询完成：** CPU 轮询完成队列，确认远程写入完成。
5. **恢复流：** 更新共享内存标志，解除 CUDA 流的阻塞。



M2N Receiver 设计： 接收端同样采用 CPU 辅助（下图 Figure 7）：

6. **等待缓冲区空闲：** 确保 GPU 不再使用接收缓冲区。
7. **阻塞流：** 防止 GPU 读取未就绪的数据。
8. **轮询与刷新：** 轮询网络完成事件，并使用 GDRCopy 进行 Flush 操作，确保数据一致性。
9. **恢复流：** 通知 GPU 数据已就绪。



优化技术：

- **消除 GPU-CPU 拷贝：** 利用 GPUDirect 技术，数据直接在 GPU 间传输，无需经过 CPU 内存中转。
- **高优先级 ACK：** 将确认包放入高优先级队列，避免被数据包阻塞，减少控制面延迟。
- **拥塞控制调优：** 针对 M2N 通信中数据量不均衡的特点，调整拥塞控制算法，加速收敛。

5. 实验设置

5.1. 数据集

来源与规模： 实验使用的数据集来自字节跳动的生产环境。这是一个真实世界的 LLM 服务请求集合。**特点：**

- **输入/输出长度：** 输入序列的中位长度为 571 个词元，输出序列的中位长度为 159 个词元。这反映了典型的对话或生成场景。
- **选择原因：** 使用生产数据集能真实反映系统在实际负载下的性能表现，特别是请求分布和序列长度特征对 KV 缓存和批处理效率的影响。

5.2. 评估指标

论文主要关注以下指标：

1. 每 GPU 解码吞吐量

- **概念定义：** 衡量系统在单位时间内生成的词元数量，并归一化到每个 GPU 上。这是评估推理系统效率的核心指标，直接关系到服务成本。
- **数学公式：**

$$\text{Throughput} = \frac{\text{Total Tokens Generated}}{\text{Total Time} \times \text{Number of GPUs}}$$

- **符号解释：**
 - Total Tokens Generated: 实验期间生成的所有输出词元总数。
 - Total Time: 实验运行的总时间。
 - Number of GPUs: 参与推理的 GPU 总数。

2. 单位成本吞吐量

- **概念定义：** 将吞吐量除以硬件成本，用于衡量系统的经济效益。在异构部署场景下尤为重要。
- **数学公式：**

$$\text{Cost-Normalized Throughput} = \frac{\text{Throughput}}{\text{Total Cost}}$$

- **符号解释：**
 - Total Cost: 部署所有 GPU 的总成本（通常以相对价格表示，如以某款 GPU 价格为基准 1.0）。

3. 时间间隔 / 延迟

- **概念定义：** 生成一个输出词元所需的平均时间。反映了用户感知的响应速度。
- **数学公式：**

$$\text{TBT} = \frac{\text{Total Decoding Time}}{\text{Total Output Tokens}}$$

- ****符号解释：****
 - Total Decoding Time: 解码阶段的总耗时（不含预填充）。
 - Total Output Tokens: 生成的输出词元总数。

5.3. 对比基线

论文将 MegaScale-Infer 与以下两个最先进的 LLM 服务系统进行了对比：

- **vLLM [51]:** 工业界广泛使用的开源框架，支持 PagedAttention 和连续批处理。它主要采用张量并行。
- **TensorRT-LLM [57]:** NVIDIA 推出的高性能推理框架，包含高度优化的内核。它支持张量并行、流水线并行以及专家并行。

代表性： 这两个基线代表了当前开源和工业界 LLM 推理的最高水平，具有极强的对比参考价值。

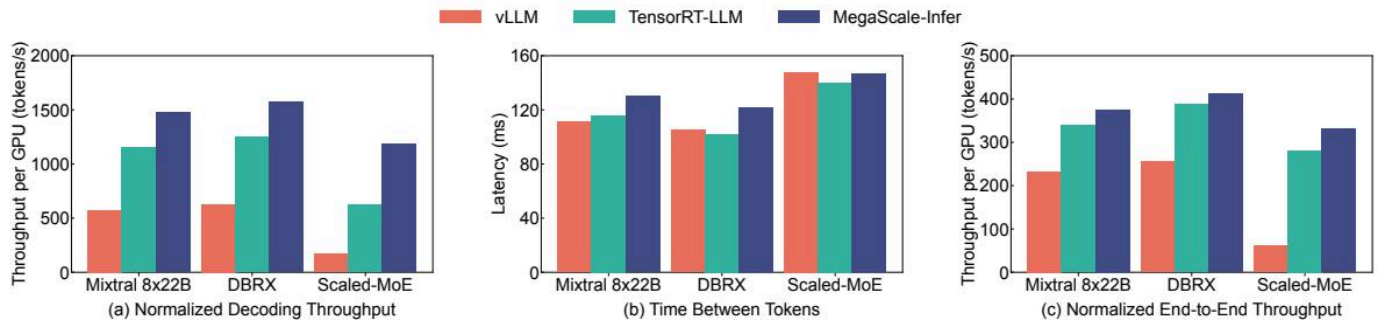
以下是原文 Table 3 的结果，展示了不同硬件的性能规格及性价比（价格已归一化）：

Accelerator	Price	Cap. (GB)	Bw. (GB/s)	Comp. (TFLOPS)	Performance	
					GB	
L20	1.00	48	864	119.5	48	8
H800	5.28	80	3430.4	989	15.2	6
A800	2.26	80	2039	312	35.4	9
H20	1.85	96	4096	148	51.9	2
L40S	1.08	48	864	362	44.4	8

6. 实验结果与分析

6.1. 核心结果分析

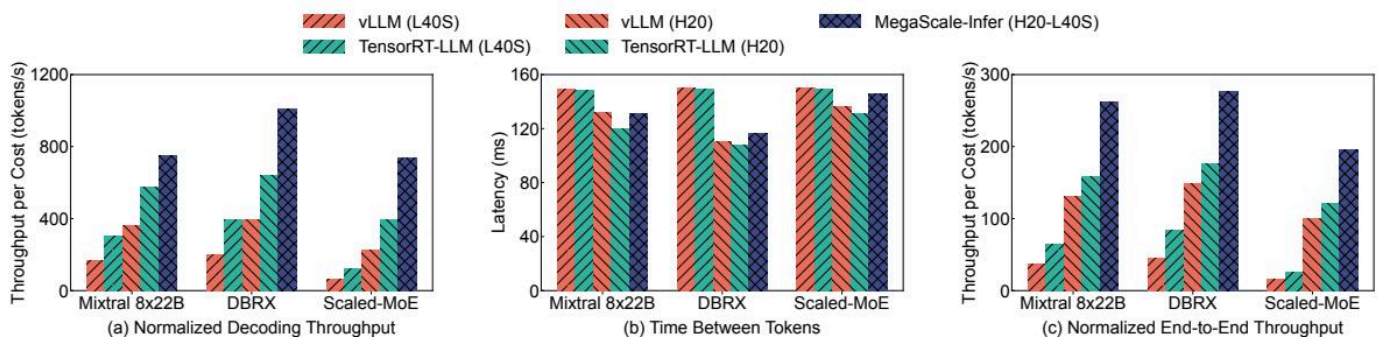
同构部署性能： 在 NVIDIA Ampere (A100/A800) GPU 集群上的实验结果如下图（原文 Figure 8）所示：



该图像是图表，展示了MegaScale-Infer在NVIDIA Ampere GPU上的性能。图中包含三个子图，分别显示了标准化解码吞吐量、令牌间延迟和标准化端到端吞吐量。不同颜色的条形图代表了不同模型的比较，包括vLLM、TensorRT-LLM、DBRX、Scaled-MoE和MegaScale-Infer。通过这些对比数据，可以直观理解MegaScale-Infer在性能上的优势。

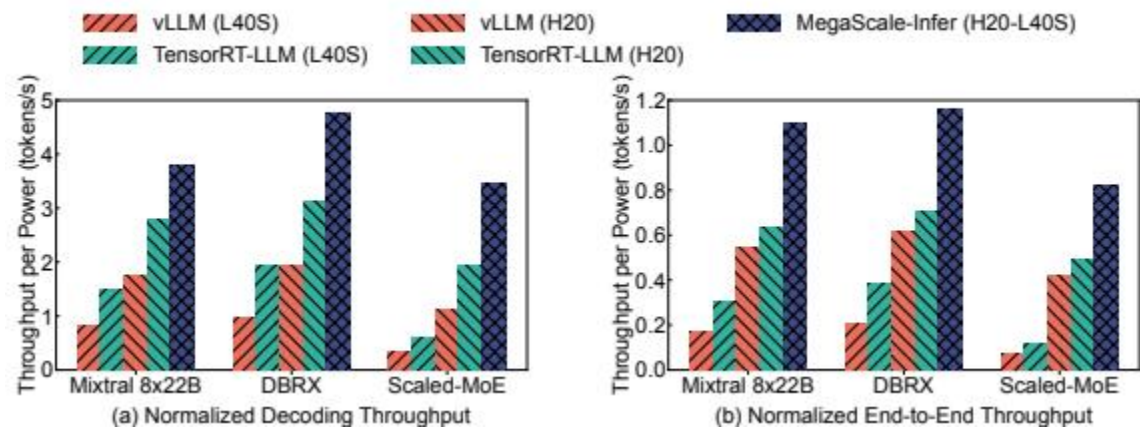
- **解码吞吐量 (Figure 8a):** MegaScale-Infer 在所有测试模型上均显著优于基线。对于 Mixtral 8x22B，其吞吐量是 vLLM 的 2.56 倍，是 TensorRT-LLM 的 1.28 倍。对于更大的 Scaled-MoE 模型，优势更加明显，分别达到 7.11 倍和 1.90 倍。
 - **分析:** vLLM 和 TensorRT-LLM 采用整体部署，MoE 的稀疏性导致每个专家处理的批次太小，无法跑满 GPU。MegaScale-Infer 通过解耦，聚合了多个注意力节点请求，使专家批次变大，从而充分利用了 GPU 算力。
- **延迟 (Figure 8b):** 尽管解耦引入了跨节点通信，但得益于乒乓流水线和高性能 M2N 库，MegaScale-Infer 的平均 TBT 与基线相当，并未因架构解耦而牺牲延迟。
- **端到端吞吐量 (Figure 8c):** 包含预填充阶段时，增益有所收窄（最高 1.18 倍），因为预填充阶段是计算密集的，解耦优势不明显。

异构部署性能: 在由 NVIDIA H20 (高带宽) 和 L40S (高算力) 组成的异构集群上的结果如下图（原文 Figure 9）所示：



该图像是图表，展示了MegaScale-Infer在不同平台上的性能比较，包括标准化解码吞吐量、令牌间延迟和标准化端到端吞吐量。数据反映了不同模型的表现，显示了在H20和L40S GPU上，MegaScale-Infer的效果相比其他方案的优势。

- **单位成本吞吐量 (Figure 9a):** MegaScale-Infer 将 H20 用于注意力，L40S 用于专家。相比在 H20 上运行基线，其单位成本吞吐量提升了 3.24 倍 (vs vLLM) 和 1.86 倍 (vs TensorRT-LLM)。
 - **分析:** 这证明了异构部署的有效性。H20 虽然算力弱，但带宽高且便宜，适合跑注意力；L40S 算力强且便宜，适合跑专家。MegaScale-Infer 充分利用了两者优势，实现了极致的成本效益。
- **功率效率 (Figure 10):** 异构部署也带来了功耗优势。下图显示 MegaScale-Infer 在解码和端到端场景下，每瓦特吞吐量均显著高于基线。



该图像是图表，展示了MegaScale-Infer在NVIDIA H20和L40S GPU上的每单位功率的吞吐量。图中比较了不同模型（如vLLM和TensorRT-LLM）的解码与端到端吞吐量，以评估其性能。图表包括两个子图，分别展示了归一化解码吞吐量和归一化端到端吞吐量的结果。

6.2. 数据呈现 (表格)

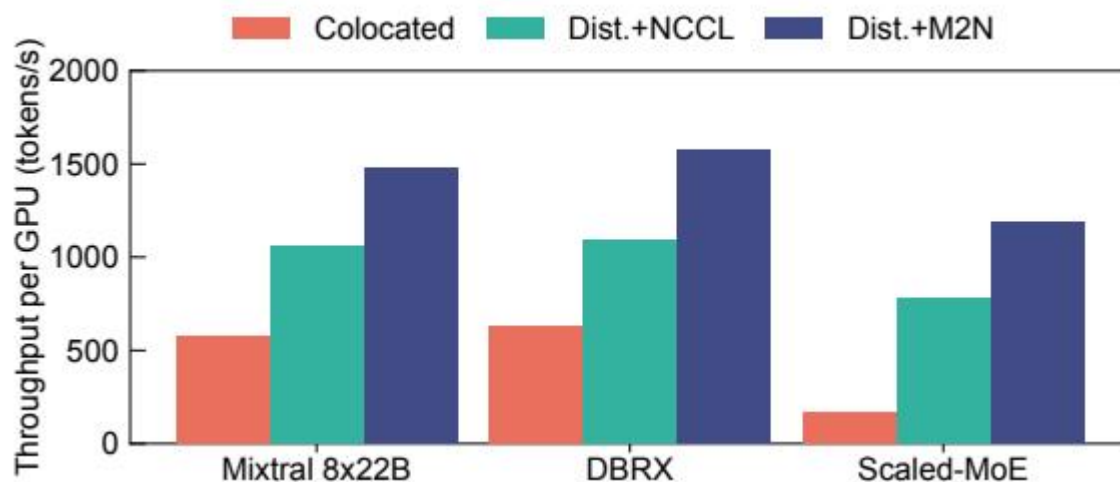
以下是原文 Table 4 的结果，列出了实验所使用的 MoE 模型配置：

Model	#Layers	Hidden Size	#Experts	top-k	Intermediate Siz
Mixtral-8×22B	56	6144	8	2	16384
DBRX	40	6144	16	4	10752
Scaled-MoE	48	8192	32	4	8192

6.3. 消融实验/参数分析

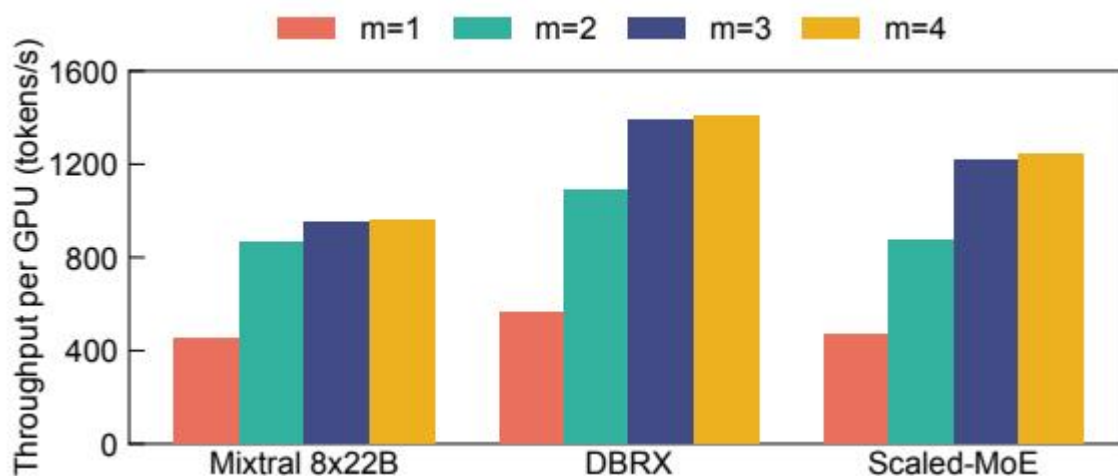
1. 解耦架构与 M2N 通信的有效性： 下图（原文 Figure 13）展示了各组件的贡献。

- **Colocated (vLLM):** 基线。
- **Dist.+NCCL:** 仅使用解耦架构但使用 NCCL 通信。吞吐量提升了 4.66 倍，说明解耦带来的批次聚合收益巨大。
- **Dist.+M2N:** 使用解耦架构 + 自研 M2N 库。吞吐量进一步提升了 1.53 倍，说明优化的通信库对于隐藏延迟至关重要。



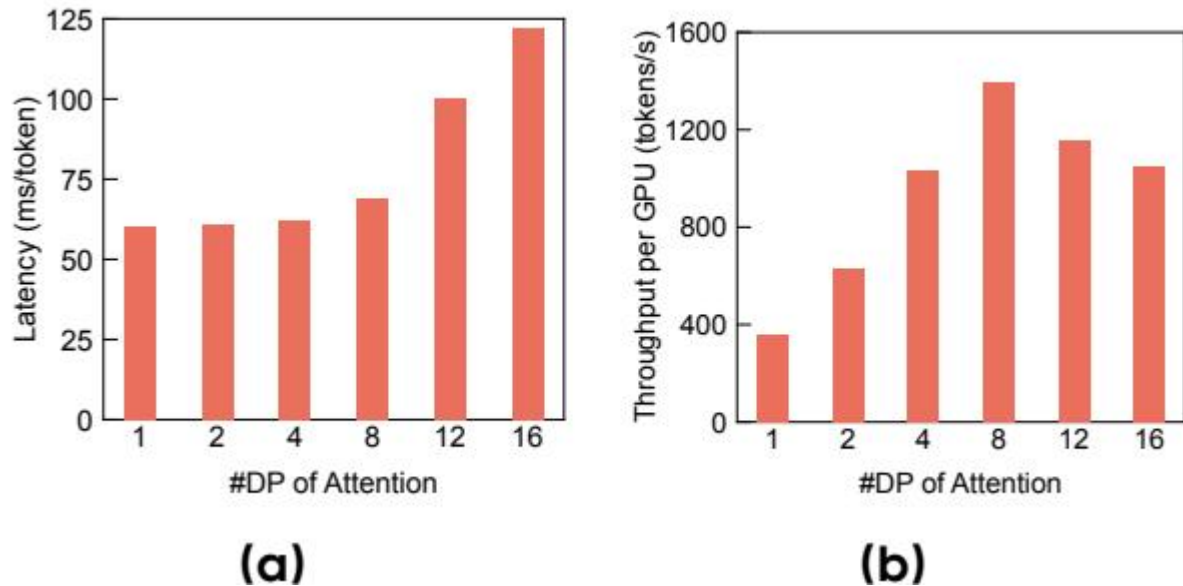
2. 微批次数量 (m) 的影响: 下图 (原文 Figure 14) 展示了不同微批次数量下的性能。

- $m = 1$ 时 (无流水线), 吞吐量最低。
- $m = 2$ 时, 引入乒乓流水线, 吞吐量大幅提升 (约 1.9 倍)。
- $m = 3$ 时, 进一步增加微批次以覆盖通信开销, 吞吐量继续提升。
- $m > 3$ 时, 收益边际递减。论文通常选择 $m = 3$ 或 4。



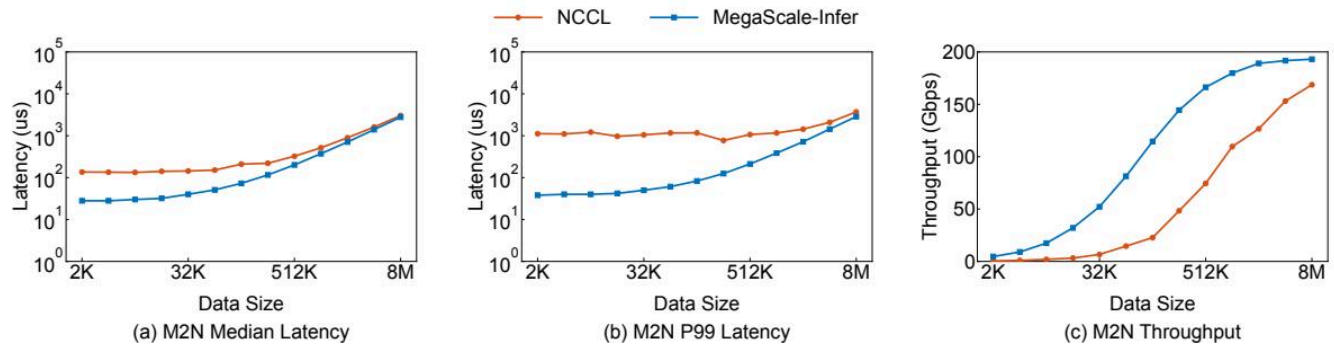
3. 部署计划 (DP 度) 的影响: 下图 (原文 Figure 15) 展示了注意力节点数据并行度对性能的影响。

- 随着 DP 度增加, 专家端接收到的批次变大, 计算时间 T_e 增加。
- 当 DP 度较小时, 专家端是瓶颈; 当 DP 度较大时, 注意力端成为瓶颈。
- 只有在 DP 度适中 (图中 DP=8) 时, $T_a \approx T_e$, 系统达到最优吞吐量。这验证了部署计划搜索算法中负载均衡约束的重要性。



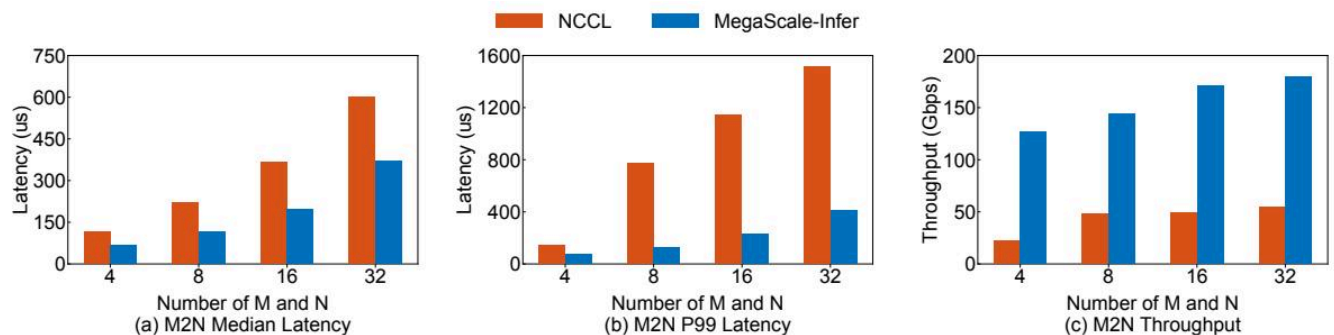
4. M2N 通信性能微基准测试： 下图（原文 Figure 11 和 12）对比了 M2N 库与 NCCL 的性能。

- **延迟：** M2N 的中位延迟和 P99 延迟均远低于 NCCL，特别是在数据量较小或规模较大时，优势更明显。
- **吞吐量：** M2N 的吞吐量显著高于 NCCL，最高可达 9.9 倍。



该图像是图表，展示了不同数据大小下的M2N通信性能，包括中位延迟、P99延迟和吞吐量。

图中蓝色线条代表MegaScale-Infer，橙色线条代表NCCL，整体趋势显示MegaScale-Infer在大数据量下表现出色。



该图像是图表，展示了在不同发送者 (M) 和接收者 (N) 数量下，MegaScale-Infer与NCCL的M2N通信性能。包括(a) Median Latency, (b) P99 Latency, 和(c) Throughput, 结果表明MegaScale-Infer提供了更低的延迟和更高的吞吐量。

7. 总结与思考

7.1. 结论总结

MegaScale-Infer 针对大规模 MoE 模型推理中的稀疏性瓶颈，提出了一种系统级的解决方案。通过解耦注意力与 FFN 模块，系统实现了计算资源的独立扩展和异构硬件部署。配合乒乓流水线并行和定制化的 M2N 通信库，成功隐藏了跨节点通信开销，将受限于内存的 FFN 转化为计算密集型任务。实验证明，该方法在吞吐量、延迟和成本效益上均取得了显著提升，已在字节跳动的生产环境中部署并降低了 1.5-2.0 倍的服务成本。

7.2. 局限性与未来工作

局限性：

- **系统复杂性：** 解耦架构和自定义通信库增加了系统的部署和维护复杂度。
- **单次迭代延迟：** 虽然平均延迟相当，但流水线机制可能会略微增加单个请求的首个词元延迟（尽管文中未明确强调，这是流水线的固有特性）。
- **负载均衡依赖：** 性能高度依赖于专家负载的均衡性。如果某些专家极热，可能导致负载不均。

未来工作：

- **动态调度：** 进一步优化专家负载均衡策略，应对更动态的工作负载。
- **更广泛的硬件支持：** 将异构部署扩展到更多类型的加速器（如 TPU 或其他定制芯片）。

7.3. 个人启发与批判

启发：

1. **硬件感知设计：** 本文最大的启发在于“让架构适应硬件”。通过识别不同计算模块（Attention vs FFN）对硬件资源（带宽 vs 算力）的不同需求，设计出解耦架构，从而能像搭积木一样选择最合适的硬件，这是未来 AI 系统设计的重要方向。
2. **通信即瓶颈：** 在分布式系统中，通信往往是隐形杀手。MegaScale-Infer 没有满足于使用现成的 NCCL，而是针对特定模式（M2N）深度优化，甚至不惜用 CPU 辅助以节省 GPU 资源，这种极致的性能优化精神值得学习。
3. **稀疏性的双刃剑：** MoE 稀疏性虽然降低了理论计算量，但给系统实现带来了巨大的负载不均衡挑战。这提醒我们，算法层面的优化必须配合系统层面的支持，才能落地生效。

批判与思考：

- **M2N vs DeepEP:** 论文提到 DeepEP 使用 GPU 直通通信，而 MegaScale-Infer 使用 CPU 代理。这引发了思考：在极端大规模场景下，当连接数极多且数据包极小时，CPU 的中断处理和轮询是否会成为瓶颈？此时 DeepEP 的 GPU 并行处理能力可能更优。因此，MegaScale-Infer 的方案可能在当前的数据包大小（几百 KB）下是最优的，但未来可能需要混合策略。
- **解耦的普适性：** 这种解耦思想是否可以应用于其他模块？例如，将 MoE 的门控网络也单独解耦出来？或者应用于 Transformer 以外的架构？这为后续研究提供了广阔的空间。